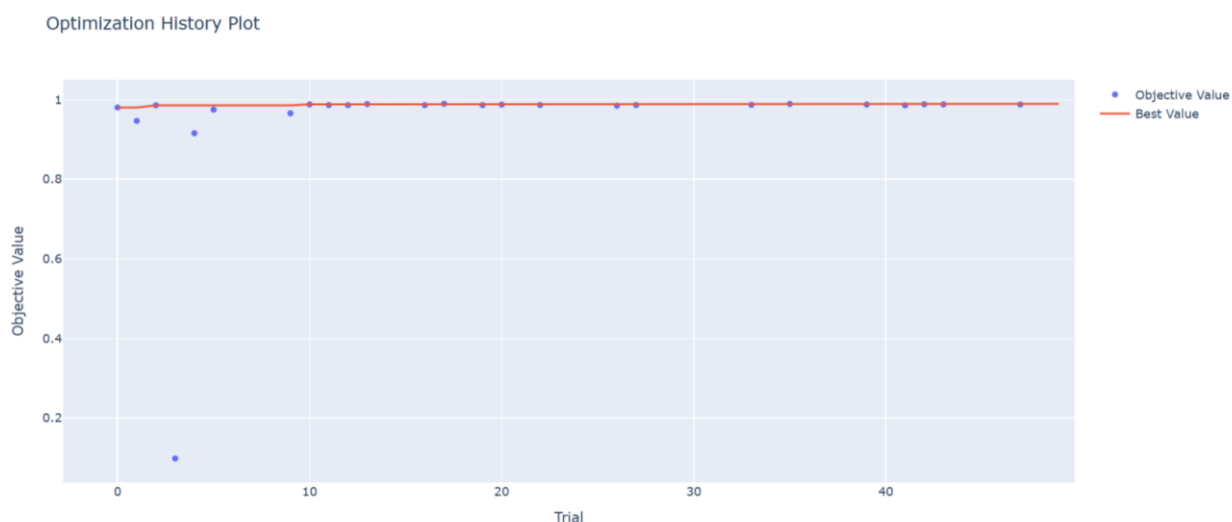


Nasz projekt polegał na analizie działania sieci neuronowych oraz optymalizacji ich parametrów dla zadania klasyfikacji obrazów ze zbioru MNIST. W pierwszej części wykorzystano bibliotekę Optuna do automatycznego doboru hiperparametrów konwolucyjnej sieci neuronowej CNN. Dla każdej wygenerowanej konfiguracji model był trenowany przez określoną liczbę epok, a po każdej epoce obliczana była dokładność na zbiorze walidacyjnym. Wyniki były przekazywane do Optuny, która na ich podstawie wybierała najbardziej obiecujące kombinacje parametrów do kolejnych prób. Po wyznaczeniu najlepszej konfiguracji model został ponownie wytrenowany na pełnym zbiorze treningowym i oceniony na zbiorze testowym. Dodatkowo zaimplementowano interfejs użytkownika w bibliotece Gradio, umożliwiający wgrywanie lub rysowanie cyfr i ich klasyfikację przez wytrenowany model. W celu zwiększenia interpretowalności wyników wykorzystano bibliotekę SHAP, która pozwala wizualizować wpływ poszczególnych pikseli obrazu na końcową decyzję sieci neuronowej.

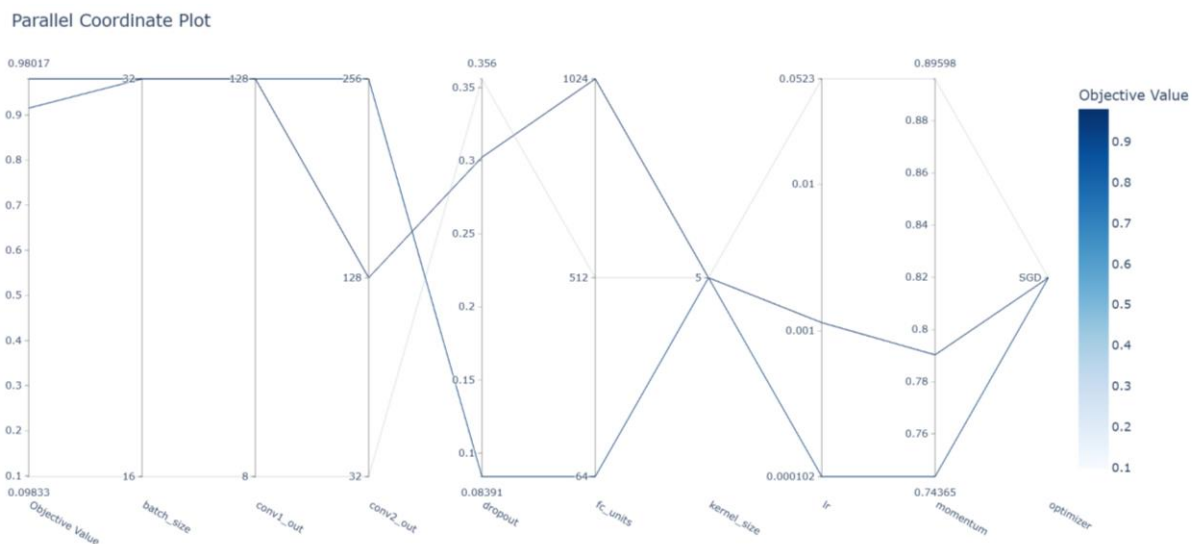
Po napisaniu pierwszej wersji kodu postanowiliśmy przetestować możliwie szeroki zakres hiperparametrów. Celem było określenie, które wartości mają największy wpływ na skuteczność modelu oraz zawężenie zakresów do dalszych testów.

Testowane były następujące hiperparametry:

- kernel_size: 3, 5, 7 (rozmiar kernela splotu)
- conv1_out: 8, 16, 32, 64, 128 (liczba filtrów w pierwszej warstwie Conv)
- conv2_out: 16, 32, 64, 128, 256 (liczba filtrów w drugiej warstwie Conv)
- fc_units: 64, 128, 256, 512, 1024 (liczba neuronów w warstwie Fully Connected)
- dropout: 0.0–0.7 (współczynnik Dropout)
- batch_size: 16, 32, 64, 128, 256 (rozmiar batcha)
- optimizer: Adam, SGD, RMSprop (optymalizator)
- lr: 0.00001–0.1 (learning rate)
- momentum: 0.5–0.99 (tylko dla SGD)
- n_trials = 50
- n_epochs = 3



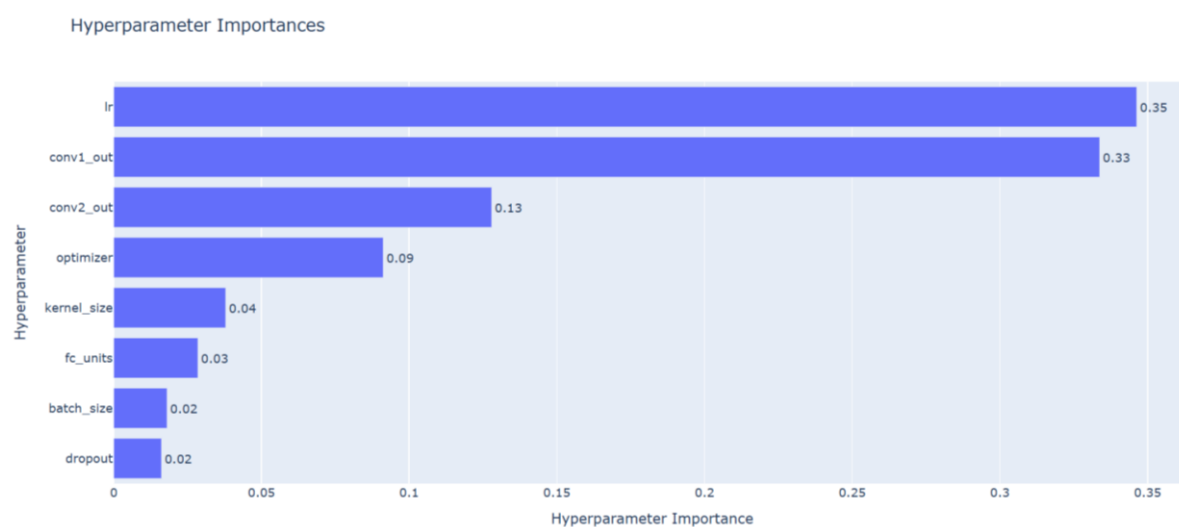
Wykres optimization_history przedstawia zmianę najlepszej uzyskanej dokładności w kolejnych trialach Optuny. Można zauważyć, że największa poprawa następuje w pierwszych 15-20 próbach, a dalsze trial nie przynoszą istotnego wzrostu jakości modelu. Oznacza to, że dla badanego problemu do znalezienia dobrej konfiguracji hiperparametrów wystarcza około 15-20 triali, co pozwala skrócić czas optymalizacji.



Wykres parallel_coordinate przedstawia zależności między wartościami hiperparametrów a uzyskaną skutecznością modelu. Każda linia odpowiada jednej testowanej konfiguracji parametrów. Analiza przebiegu linii pozwala określić, które kombinacje hiperparametrów prowadziły do lepszych wyników. Wykres został wykorzystany głównie do pokazania możliwości analitycznych biblioteki Optuna i nie będzie brany do uwagi na końcu tego etapu.

Linia	Objective Value	Batch Size	Conv1 Out	Conv2 Out	Dropout	FC Units	Kernel Size	Learning Rate (lr)	Momentum	Optimizer
1	0.0983	16	8	32	0.3560	512	5	0.052335	0.895979	SGD
2	0.9802	32	128	128	0.3024	1024	5	0.001140	0.790283	SGD
3	0.9155	32	128	256	0.0839	64	5	0.000102	0.743652	SGD

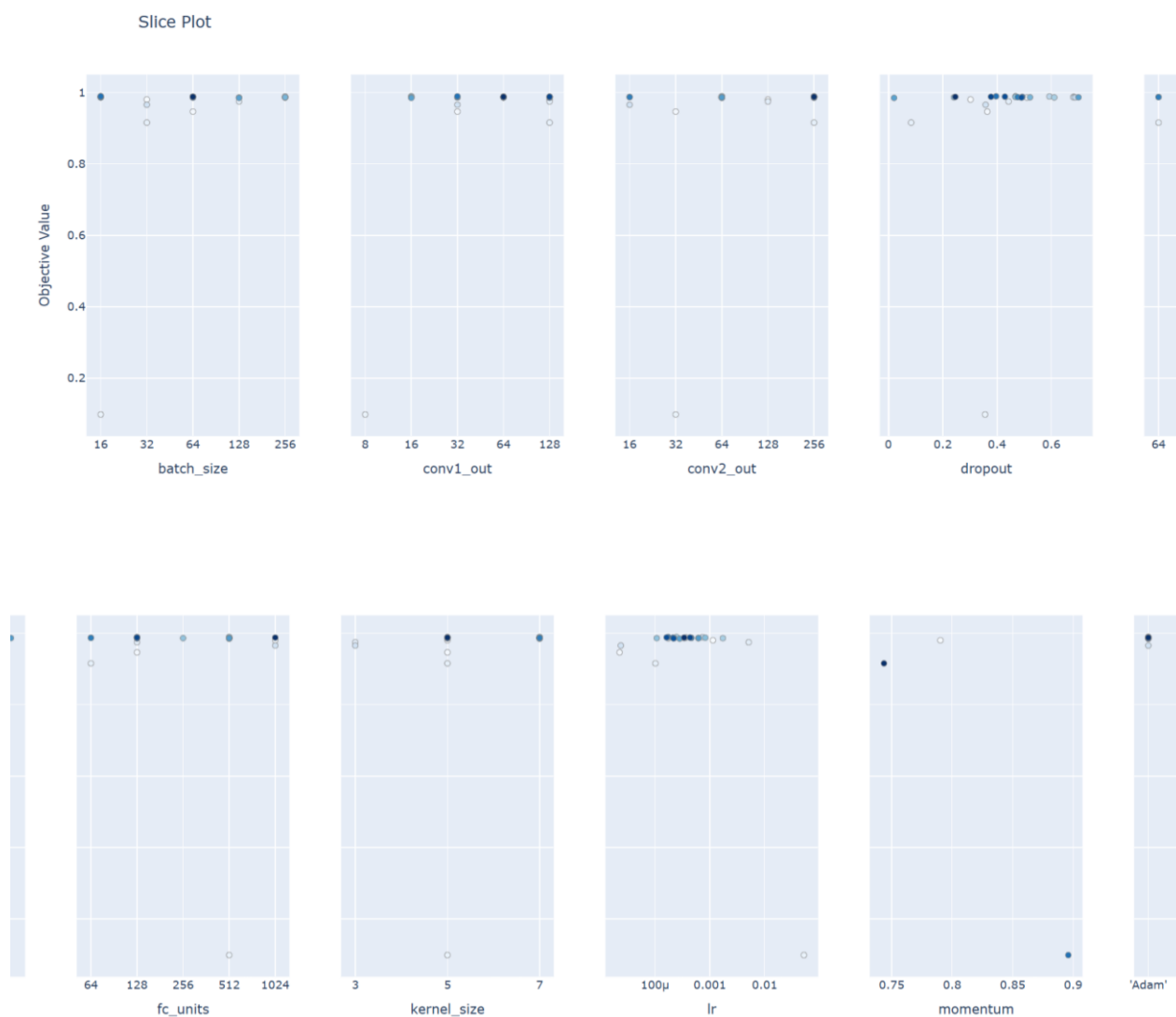
Najlepszy wynik (0.9802) uzyskano dla średniego dropout (~0.30), dużej liczby filtrów (128/128), największej warstwy FC (1024) oraz learning rate około 0.001. Najgorszy wynik (0.0983) wystąpił przy bardzo wysokim learning rate (0.052335), co sugeruje, że zbyt duży krok uczenia może znacząco pogarszać skuteczność modelu.

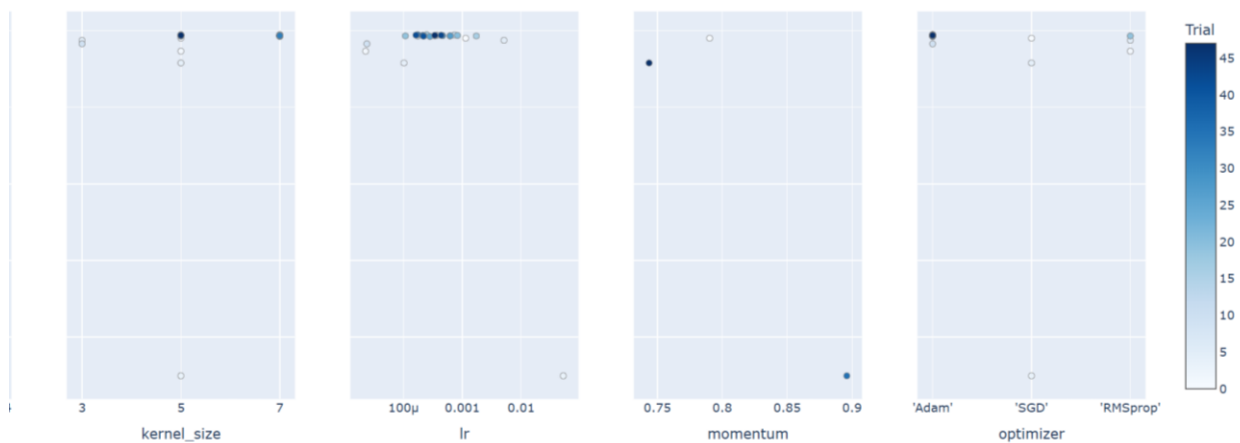


Wykres param_importances przedstawia wagę poszczególnych hiperparametrów dla końcowej skuteczności modelu. Im wyższa wartość, tym większy wpływ dany parametr miał na uzyskiwany wynik podczas optymalizacji.

Hiperparametr	Ważność
lr	0.35
conv1_out	0.33
conv2_out	0.13
optimizer	0.09
kernel_size	0.04
fc_units	0.03
batch_size	0.02
dropout	0.02

Największy wpływ na skuteczność modelu miały learning rate (0.35) oraz liczba filtrów w pierwszej warstwie konwolucyjnej conv1_out (0.33). Parametry takie jak batch_size i dropout miały najmniejszy wpływ (po 0.02), dlatego ich dokładne dostrajanie w badanym zakresie nie wpływało istotnie na końcowy wynik modelu.





Wykres slice_plot pokazuje wpływ poszczególnych wartości hiperparametrów na jakość modelu. Najlepsze wyniki były najczęściej uzyskiwane dla małego współczynnika uczenia (lr), optymalizatora Adam, batch_size 256 oraz architektury zawierającej conv1_out = 16, conv2_out = 64 i fc_units = 256.

Parametr	Najlepsza jakość	Najgorsza jakość
lr	0.00025	0.0523
conv1_out	16	8
conv2_out	64	32
optimizer	Adam	SGD
kernel_size	7	5
batch_size	256	16
dropout	0.47	0.36
momentum	0.79	0.90
fc_units	256	512

Liczby, które najlepiej pokazały się podczas testów:

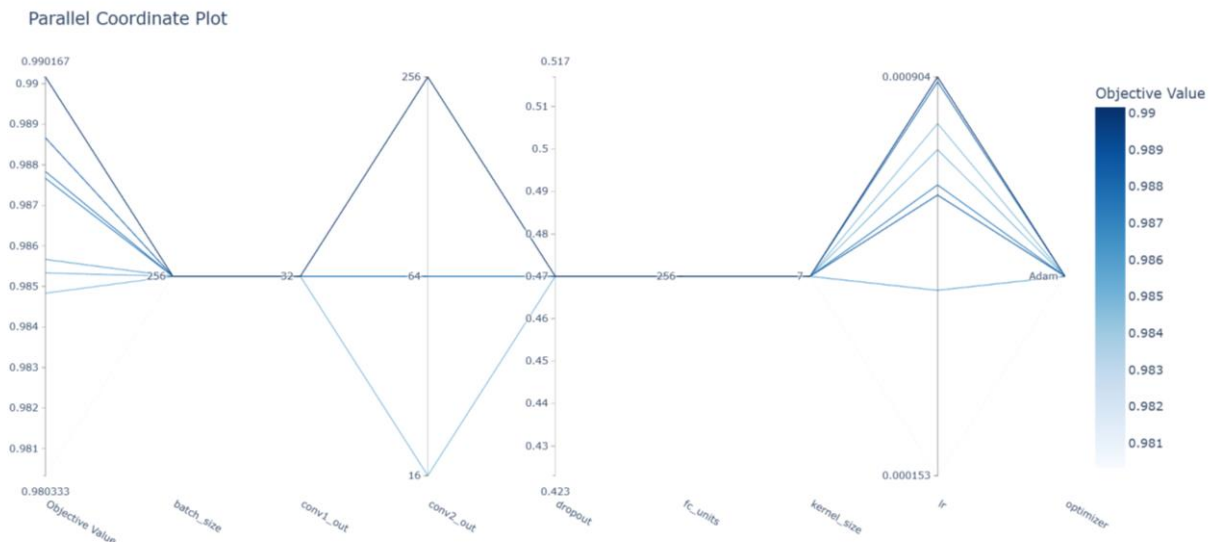
- lr: 0.0001 - 0.001
- conv1_out: 16, 32
- conv2_out: 16, 64, 256
- optimizer: Adam > RMSprop > SGD
- kernel_size: 7
- batch_size: 128, 256
- dropout: 0.4 - 0.5
- momentum: 0.78 - 0.80
- fc_units: 128, 256

Parametry do kolejnego testu

Na podstawie przeprowadzonych eksperymentów do następnej serii testów będą wykorzystane następujące zakresy:

- n_trials = 20
- n_epochs = 3
- kernel_size: 7

- conv1_out: 16, 32
- conv2_out: 16, 64, 256
- fc_units: 256 (rola była stosunkowo niewielka)
- dropout: 0.47 (nie odgrywał istotnej roli)
- batch_size: 256 (nie odgrywał istotnej roli)
- optimizer: Adam, SGD
- lr: 0.0001 - 0.001
- momentum: 0.78 - 0.80



Best validation accuracy : 0.9902

Best hyperparameters:

```

kernel_size = 7
conv1_out = 32
conv2_out = 256
fc_units = 256
dropout = 0.47
batch_size = 256
optimizer = Adam
lr = 0.000904358621560433

```

Retraining best config on full training set ...

Epoch	TrainLoss	TrainAcc
1	0.2148	0.9339
2	0.0556	0.9834

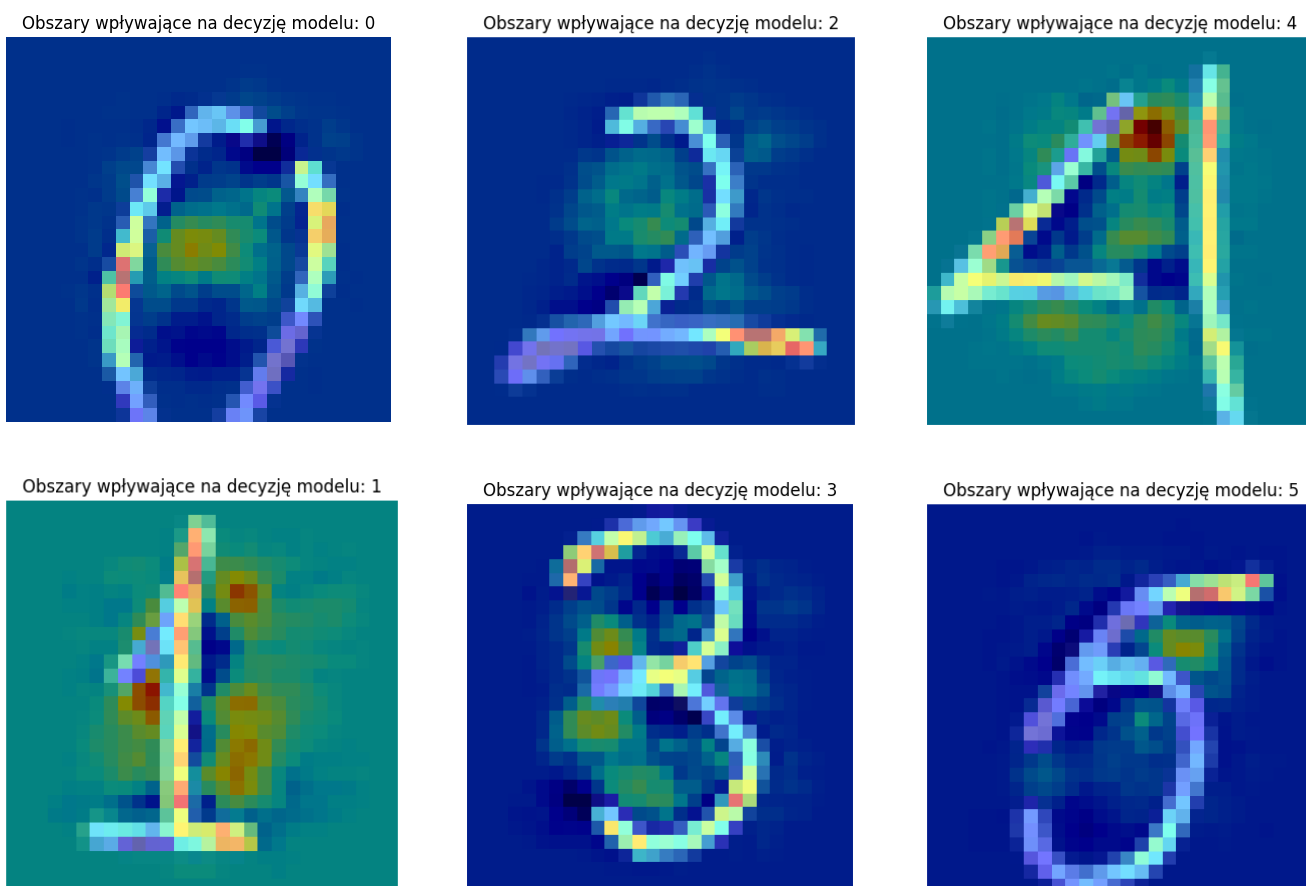
Test loss : 0.0300

Test accuracy : 0.9902 (99.02%)

Końcowa optymalizacja hiperparametrów pozwoliła uzyskać dokładność walidacyjną na poziomie 99,02%. Najlepsza konfiguracja modelu obejmowała parametry: kernel_size = 7, conv1_out = 32, conv2_out = 256, fc_units = 256, dropout = 0,47, batch_size = 256, optymalizator Adam oraz współczynnik uczenia 0,000904. Wykres optuna_parallel_coordinate pokazuje, że najwyższe wartości funkcji celu były osiągnięte dla stosunkowo małych wartości learning rate, jednak nie były to wartości skrajnie małe, co i było zakładane we wcześniejszych testach. Widoczne jest również, że lepsze wyniki uzyskiwano przy większej liczbie filtrów w drugiej warstwie konwolucyjnej oraz przy wykorzystaniu optymalizatora Adam. Po ponownym wytrenowaniu modelu z najlepszym zestawem parametrów uzyskano końcową dokładność na zbiorze testowym równą 99,02%, przy wartości funkcji straty 0,0300, co potwierdza bardzo wysoką skuteczność modelu klasyfikującego obrazy MNIST.

Następnie stworzono końcową wersję kodu, w której wykorzystano biblioteki Gradio oraz SHAP. Gradio odpowiadało za przygotowanie interfejsu użytkownika umożliwiającego rysowanie oraz prezentację wyników klasyfikacji w formie graficznej. Biblioteka SHAP została wykorzystana do wizualizacji wpływu poszczególnych pikseli obrazu na decyzję modelu. Należy zwrócić uwagę, że model był trenowany na zbiorze MNIST, w którym cyfry są przedstawione jako białe znaki na czarnym tle. W związku z tym podczas rysowania cyfr w interfejsie konieczne było odwrócenie kolorów obrazu przed przekazaniem go do modelu. Odwrócenie kolorów obrazu zostało zrealizowane przy użyciu funkcji: `original_image = transforms.functional.invert(original_image)`

W wizualizacjach SHAP jaśniejsze kolory oznaczają piksele, które zwiększają prawdopodobieństwo przypisania obrazu do danej klasy, natomiast ciemniejsze obszary wskazują piksele zmniejszające pewność modelu. Analiza map SHAP pokazała, że sieć neuronowa podejmowała decyzje głównie na podstawie charakterystycznych elementów kształtu cyfr, takich jak pionowe i poziome kreski, łuki oraz zamknięte pętle.



Cyfra 0 - Mapa SHAP dla cyfry 0 pokazuje, że model zwracał uwagę na zewnętrzne krawędzie tworzące zamknięty okrąg. Wpływ miały lewe i prawe fragmenty obwodu oraz górna część cyfry. Środek cyfry miał znacznie mniejsze znaczenie dla klasyfikacji. Oznacza to, że model rozpoznawał przede wszystkim zamknięty kształt cyfry 0.

Cyfra 1 - W przypadku cyfry 1 największy wpływ na decyzję modelu miała pionowa kreska znajdująca się w centralnej części obrazu. Widoczny jest również wpływ dolnej podstawki oraz górnego fragmentu cyfry. Model skupiał się głównie na prostym pionowym kształcie.

Cyfra 2 - Dla cyfry 2 największe znaczenie miały dolna pozioma kreska oraz górne zakrzywienie tworzące kształt cyfry. Widoczne jest również znaczenie środkowej części łączącej górę z dolną podstawą. Model wykorzystywał elementy odróżniające cyfrę 2 od podobnych cyfr, takich jak 7 lub 3.

Cyfra 3 - Mapa SHAP dla cyfry 3 wskazuje, że model koncentrował się głównie na górnym i dolnym łuku. Wpływ miała również środkowa część obrazu, która odpowiada za połączenie obu łuków.

Cyfra 4 - W przypadku cyfry 4 model zwracał uwagę na pionową kreskę po prawej stronie, ukośną kreskę po lewej stronie oraz poziomą poprzeczkę przebiegającą przez środek cyfry. Są to elementy identyfikujące cyfrę 4 i odróżniające ją od pozostałych klas.

Cyfra 5 - Dla cyfry 5 największy wpływ na decyzję modelu miała górna pozioma część cyfry, środkowe załamanie oraz dolny łuk. Widoczne jest skupienie modelu na elementach konturu, które pozwalają odróżnić cyfrę 5 od cyfr 6, 8 lub 9.